Software Construction (L+E) HS 2024

Assignment 3
Week 09

Instructor: Prof. Dr. Alberto Bacchelli
Teaching Assistants: Konstantinos Kitsios, Nils Reusch, Izia Wang

Overview

In this assignment you will implement your own version control system! Inspired by `git`, your tool, named `tig`, will support file versioning, file status management, and retrieval of past file states.

Process Requirements

To correctly complete this assignment, you **must**:

1. **Work with your group only** (unless otherwise stated). You will work in the groups formed during assignment 01. While you may discuss ideas with other groups, each group is expected to develop its own unique solution. A strict plagiarism policy will be enforced for all submitted work.

2. **Work using Git and Gitlab.** You are required to use Git and GitLab (hosted at `gitlab.uzh.ch`) for version control and collaboration within your group. Follow this process:

   a) **Repository.** Use the GitLab repository you created for assignment 01 at gitlab.uzh.ch.

   b) **Assignment folder.** For this specific assignment, create and work on a folder named: `[groupname on OLAT]-a[AssignmentNumber]`.[1] The folder **must contain at least** the following elements (also summarized in Figure 1):

      - a `README.md` file that explains the decisions taken in the source code and demonstrate the usage of your tool. It should also include any use of Generative AI (see point 'Use of Generative AI' below);
      - a `repository.md` file that contains only the link to the GitLab repository of your group;
      - a `tig.py` file that contains the implementation of your version control system in Python;
      - a `Tig.java` file that contains the implementation of your version control system in Java;
      - a `Tig.class` file that is the compiled version of `Tig.java`;
      - a `repo/` folder that you will create in Step 01;

[1] *e.g.*, a correct name would be: `SoCo_HS24-group_01-a3`

- a `repo_java/` folder that you will create in Step 02;
- (Optional) any additional Python or Java files that could make the code cleaner. For example, you may want to separate the command-line argument parsing from the main logic.

c) **Version Control with Git.** All code and documents must be managed through Git. Regular commits should be made to the repository. Commit messages should be descriptive and clear. A single large commit at the end of the assignment will not be accepted.[2]

d) **Use of Merge Requests.** You are encouraged to use merge requests when developing and integrating changes. This helps track who contributed to specific features and encourages peer review within the group. Each merge request should include a description of the changes made and a reference to the corresponding task being addressed.

e) **Contribution Tracking and Fairness.** We will analyze the Git history at the end of the assignment to ensure that all group members have contributed fairly. Ensure that all contributions are committed using the correct author details in Git (do not share accounts). Any attempt to manipulate the real author of changes or the history of Git to obscure or misrepresent contributions will be considered a violation of academic integrity.

Figure 1: Folder Structure

```
SoCo_HS24-group_01-a3/
├── README.md
├── repository.md
├── tig.py
├── Tig.java
├── Tig.class
├── repo/
│   └── ...
└── repo_java/
    └── ...
```

3. **Use of Generative AI.** While you may use tools like Generative AI for assistance, you must ensure that the final solution is your own original work. Generative AI should only be used as a tutor or assistant (e.g., to help troubleshooting or learning a new API), not for directly producing the solutions. If you make use of AI, **you have to document all the prompts you have used and an overall explanation of how you used Generative AI**. To avoid misuse of Generative AI, we will do the following:

[2] Have a look here for the benefits of frequent and atomic commits.

a) **Random audits**: Ten groups will be randomly chosen to present and explain the reasoning behind their code in detail. This will include the rationale for specific design choices. Failure to demonstrate a thorough understanding of the submitted code may result in penalties or disqualification.

b) **Plagiarism detection**: Automated tools will be used to detect code similarities and identify AI-generated content. Any instances of code reuse or reliance on AI beyond the limits described above will be flagged for further review.

4. **Follow the instructions and submission guidelines precisely**. Ensure that your solutions adhere to the structure outlined in the submission section.

5. **Deadline.** The deadline for this assignment is **Dec 03, 2024 @ 18:00**.

Submission

To submit your assignment, closely follow these steps:

- Before the deadline, finalize your work by making a last commit to the `main` branch of your GitLab repository. Tag this commit using an annotated tag[3] named `assignment3_submission`.
- Then, download your assignment folder (including the folder itself) and upload to the OLAT task by the deadline.

Best Practices

Some key strategies to help you **succeed**:

- **Start early:** The earlier you start, the more time you will have to address any challenges.
- **Break the tasks down:** Tackle the assignment step by step. Focus on completing one task at a time to avoid feeling overwhelmed.
- **Refer back to the lectures and the textbook:** If you find yourself stuck, revisit the lectures and the textbook. Everything you need to know for this lab has been covered in class.

[3] `https://docs.gitlab.com/ee/user/project/repository/tags/`

A Brief Introduction to Version Control Systems

In a version control system like `git` (or our simplified version, `tig`), files can exist in several *states* based on their tracking status and changes:

1. **Untracked**: A file that exists in the project directory but has not yet been added to the version control system is considered *untracked*. This means the system is currently ignoring it, and it won't be saved in any future commits until it is explicitly staged (using the `add` command below).

2. **Modified**: A file that is not *untracked* and has changed since the last commit is considered *modified*.

3. **Staged**: A that is ready to be committed enters the *staged* state by running the `add` command. It can come either from the *untracked* or the *modified* stage.

4. **Committed**: Once a staged file is committed (using the `commit` command below), it is saved permanently in the repository's history. Each commit acts like a checkpoint, storing all staged files with a unique identifier, a timestamp, and a commit message written in natural language, allowing you to restore the project to that exact state in the future.

By managing files in these states, a version control system allows you to track changes, save specific versions of your project, and revert to previous versions if needed. For more details, check out the official git documentation.[4]

Step 01: `tig` in Python

You are asked to implement `tig`, your own version control system in Python. To do this, extend the file archiver implementation from Chapter 10: A File Archiver of the book.[5] This will allow you to only save the contents of a file if necessary; simply duplicating the contents of all the files when only one file is changed in a commit is not an acceptable solution.

Implementation

`tig` should support the following bash commands.

- `python tig.py init <dir>`: initialize a version control repository in the specified directory. This command should create a `.tig/` folder where the repository's data (e.g., manifest files and backup files) will be stored.

- `python tig.py add <filename>`: move `<filename>` to the *staged* state. Only *untracked* or *modified* files can be moved to the *staged* state. If the file was previously *untracked*, this command will make it tracked.[6]

[4] The states in the figure are equivalent to the ones we present here, but we found it easier to think of the stages as presented here for the solution.
[5] *Hint*: Probably a single manifest file is not enough here.
[6] *Bonus*: support multiple filenames and folder names.

- `python tig.py commit <commit_message>`: commit all staged files by moving them to the *committed* state. Each commit should hold at least the following information: commit ID (unique), commit date, commit message.
- `python tig.py log -N`: print the commit ID, commit date, and commit message of the last `N` commits. If `-N` is not given, the default `N=5` is used.
- `python tig.py status`: print the current status of each file in the working directory, indicating if they are *untracked*, *modified*,[7] *staged*, or *committed*.[8]
- `python tig.py diff <filename>`: compare the current version of the file with its last committed version. Print the differences in a unified diff format.[9]
- `python tig.py checkout <commit_id>`: restore the directory's files to the state of a specific commit ID.

Use Case

After implementing `tig`, you should be able to run the following commands from the assignment's root folder:

1. Create a directory and initialize a `tig` repository:

```
mkdir repo
python tig.py init repo
```

2. Create two new files in the repository. Their status should be *untracked*:

```
cd repo
echo "Initial content" > file.txt
echo "Initial content of the other file" > other_file.txt
python ../tig.py status
```

3. Start tracking the files. Their status should be *staged*:

```
python ../tig.py add file.txt
python ../tig.py add other_file.txt
python ../tig.py status
```

4. Commit the files. Their status should be *committed*:

[7] *modified* can be further categorized into a) *modified and staged* and b) *modified and not staged*. You can either treat them as one status or two.
[8] `git` does not print all the committed-and-unmodified files, but `tig` is better :)
[9] *Bonus*: support multiple filenames.

```
python ../tig.py commit "Initial commit"
python ../tig.py status
```

5. Modify one file. It's status should be *modified*:

```
echo "Updated content" >> file.txt
python ../tig.py status
```

6. Check the difference since the last commit:

```
python ../tig.py diff file.txt
```

7. Stage and commit the modified file:

```
python ../tig.py add file.txt
python ../tig.py commit "Updated content in file.txt"
```

8. Reset (checkout) the repo to the first commit, making `file.txt` go back to it's original content:

```
python ../tig.py log
python ../tig.py checkout <commit_id_of_first_commit>
```

9. Verify that `file.txt` is in the *modified* state and `other_file.txt` is in the *committed* state:

```
python ../tig.py status
```

Include the output of these commands in your README.md file and also include the `repo/` folder in the state it has after running the above nine commands.

If you are familiar with `git`, you may notice that the above commands have many optional parameters and usages in `git` (e.g., the add command). *If your implementation correctly supports the above use case, you are eligible for the full points of the assignment.* If you are curious however, you can try supporting more features.

Step 02: `tig` in Java

Implement the functionality of `tig` in Java. Replicate the same behavior and adapt the bash commands for Java: instead of `python tig.py <command>` use `java Tig <command>` after you have compiled your Java program. Use the podcast and materials of Session 07 as a reference on how to compile and run Java files. Also, a file archiver was implemented during the same session; feel free to use it as a reference, especially for saving and staging files in a manifest format.

- What challenges did you identify during this Python to Java "translation"? What steps did you take to take to overcome them? If there is no "1-1" translation in a component, you can freely select the way to proceed and elaborate your decision in the `README.md` file.
- Implement an additional functionality that does not exist in the python implementation:
  - Add support for a `.tigignore` file.[10]
  - If a file named `.tigignore` exists in the same directory as the `.tig/` folder, read it's lines.
  - Treat each line as a filename that should be ignored by `tig`, i.e., it should not be affected by any `tig` command.

Run the equivalent use case of Python's use case and report each command's output in the `README.md` file. Name your folder `repo_java/` instead of `repo/` and include it in the submission as well.

[10] Read about `.gitignore` here.

Grading Criteria

- **Criteria:** Your assignment will be graded based on the following criteria:
  - Meeting the implementation requirements described above.
  - Thoroughly documenting your decisions and use case outputs in the `README.md` file.
  - Thoroughly understanding the source code you have authored, and being ready to explain it in depth.
- **Grades:** Assignments will be scored based on two main factors: fulfillment of criteria and evidence of effort:
  - **2 out of 2:** For assignments that meet all criteria and clearly demonstrate substantial effort.
  - **1 out of 2:** For assignments that meet some criteria but still show evident effort.
  - **0 out of 2:** For assignments that lack clear evidence of effort, regardless of criteria met.